RescueNow: AI Heart of Rural India

The problem that NO ONE cared in India.

The origin of **RescueNow** wasn't a boardroom or a hackathon brainstorming session; it was a conversation in the dense forests of Kookal. During a visit to this remote region, I asked a local how they managed medical emergencies. His answer was chilling: "We call the ambulance, but because of the roads and the weather, they arrive late. If the person makes it to the hospital, it's luck. If not, we believe God didn't want them saved."

This hackathon provided the opportunity to accelerate the MVP, but my goal is bigger: RescueNow is a production-grade instrument being released to the PlayStore for **100% FREE** to bridge the gap between a medical emergency and a delayed ambulance. We are building for the moments where the signal ends and the care must begin.

Most AI health tools are built for cities for stable WiFi, English-speaking users, and the luxury of time. We built RescueNow for the opposite: no connectivity, native language speakers, and emergencies measured in minutes.

Why Everything Else Falls Short

Flagship AI models share a critical limitation: they require constant internet connectivity and are designed as general-purpose chatbots rather than accessibility-first clinical tools. For frontline health workers in rural India, this creates two compounding problems.

First, connectivity fails precisely when it matters most — in the "black zones" where emergencies cluster. Second, cloud-based translation introduces dangerous linguistic nuance errors that corrupt life-saving triage accuracy. A mistranslated dosage instruction is not a UX problem. It is a patient safety crisis.

Many healthcare apps fail not because of their core technology, but because they try to do everything — sacrificing usability for an endless feature list. We took the opposite approach: building RescueNow with a singular focus on helping rural health workers navigate the Golden Hour of an emergency using structured, actionable medical protocols that run entirely offline.

The Breakthrough That Made It Possible

The emergence of Gemma 4, combined with the raw performance of llama.cpp, created a genuine opening. For the first time, a model capable of structured clinical reasoning could run on a mid-range Android phone without a server in sight.

We validated this across real devices — iQOO Neo 9 Pro, Vivo Y300, and Redmi Note 12 — spanning the hardware range that rural health workers actually carry. The goal was never a polished demo. It was something a field worker could trust when the stakes were real.

The Engine Room: How Gemma 4 Runs on a Phone

Running a model of Gemma 4's capability on mobile hardware is not possible through standard Flutter/Dart logic alone. To reach the performance that clinical use demands, we had to drop down into C++ and work directly at the hardware level. Here is how the pieces fit together.

The `.so` Libraries: Compiled Power

On Android, `.so` (Shared Object) files are compiled C/C++ binaries — the equivalent of `.dll` on Windows. RescueNow ships two:

`libllama.so` is the heavy lifter — the compiled llama.cpp core that handles raw tensor operations and ARMv8.2-a dot-product optimizations directly on the Snapdragon's ARM cores.

`libwrapper.so` is our custom glue layer. It wraps the complex llama.cpp internals into a simplified API that Flutter can understand, and holds the `llama_model*` and `llama_context*` in memory so the model stays loaded between conversations.

Both libraries live in `android/app/src/main/jniLibs/arm64-v8a/` — compiled once, bypassing the Android/Java Virtual Machine entirely on every inference call.

The C++ Wrapper: A Translator Between Worlds

The native llama.cpp API uses C++ classes that Dart cannot communicate with directly. `wrapper.cpp` solves this by exposing functions using C-linkage via `extern "C"` — the only function style that FFI understands.

When Flutter calls `native_load_model(path)`, the wrapper initialises the model, sets up the KV Cache and Sliding Window, and returns a clean success code. All the complexity stays on the C++ side. Flutter sees a simple, stable interface.

The FFI Bridge: The Tunnel Between Flutter and C++

FFI (Foreign Function Interface) is what connects the high-level Flutter code to the low-level C++ engine. We load the binary with `DynamicLibrary.open("libwrapper.so")` and map Dart function signatures directly to their C equivalents:

```
typedef native_infer_func = Int32 Function(Pointer<Utf8> prompt);
typedef DartInferFunc = int Function(Pointer<Utf8> prompt);
```

Dart strings are marshalled into C-pointers (`Pointer<Utf8>`) before crossing the bridge. In the other direction, a `NativeCallable` allows the C++ engine to call back into Dart each time a new token is generated — this is the mechanism behind the streaming text appearing word by word in the chat interface.

The entire inference pipeline runs inside a background Isolate, which means the engine can process batches without dropping a single UI frame.

Engineering the Clinical Engine

Winning at Build Time

Performance at the edge is decided before the app is ever installed. We spent significant time navigating the llama.cpp toolchain — balancing NDK versions and CMake configurations to leverage ARMv8.2-a dot-product instructions. By carefully tuning the `GGML_OPENMP` and `GGML_KLEIDIAI` compilation flags, we built a binary that saturates mobile CPU throughput without triggering thermal throttling mid-triage.

The 6GB Decision

Memory is the ultimate constraint in local LLM deployment. Through stress-testing across devices, we established a **6GB minimum / 8GB recommended** RAM requirement. This ensures the quantized Gemma 4 model (~3GB) has a stable buffer for the KV cache and Android system overhead — preventing the OS from terminating the clinical engine in the middle of a life-saving protocol.

This boundary was not chosen arbitrarily. It was the line between a reliable tool and an unreliable one.

Stop Recalculating the Past

Every time a health worker sends a new message, a naive implementation would reprocess the entire clinical context from scratch — a significant latency penalty on every turn. We eliminated this by implementing a KV Cache bridge that stores the Key and Value tensors of the system prompt after the first load.

The first conversation carries a brief warm-up. Every conversation after feels immediate. For someone managing an emergency, the difference between a 2-second response and a 20-second one is not a performance metric — it is the difference between a usable tool and an abandoned one.

Memory That Doesn't Run Out

Long triage sessions — a difficult birth, a multi-trauma road accident — can push deep into a conversation. To prevent memory exhaustion, we implemented a Sliding Window

into a conversation. To prevent memory exhaustion, we implemented a Sliding Window over llama.cpp's context management. As the session deepens, the engine preserves the system prompt and the most clinically relevant recent turns, while quietly retiring older tokens. The memory footprint stays fixed. The clinical continuity stays intact.

The Interactivity Problem Nobody Had Solved

Field testing revealed something the team had not anticipated. ASHA workers did not want a chatbot. They wanted a protocol — something they could work through step by step, physically checking off actions as they completed them.

The problem: LLM outputs are fluid text. There is no reliable way to map a UI button to a specific word in a generated paragraph. Tapping "Apply pressure" on screen needed to toggle exactly that item and nothing else, regardless of what else the model had generated around it.

We solved this with **Absolute Line-Indexing**. Every checklist item generated by the model is mapped to its exact physical line number in the source text. When a worker taps a step, the app targets that precise coordinate and toggles it — reliably, regardless of surrounding content. The protocol becomes genuinely interactive, not just visually suggestive.

There is also a human moment built into this system. The instant a worker checks the final item in a triage protocol, the app surfaces a quiet "Excellent Triage!" confirmation. It sounds small. In testing, field workers described it as the moment they felt the emergency had passed. Managing cognitive load and emotional state during a crisis is not a soft concern — it directly affects the quality of care that follows.

The Language Problem Was Harder Than It Looked

Early builds had a failure mode we called English-bias. Even when an ASHA worker asked a question in Telugu, the model would respond with English headers, English Mermaid node labels, and English table columns wrapped around Telugu body text. For a worker reading under pressure, this fragmentation was actively harmful.

The fix required more than translation. It required architectural intervention.

We implemented a **Linguistic Mirror Protocol** that forces the model to perform an internal language identification step before generating any output. This State-Space anchor locks the engine into the user's native script from the first token. When the question arrives in Telugu, the checklist items, the flowchart nodes, the table headers — everything renders in Telugu. The language the worker thinks in is the language the protocol arrives in.

What We Learned From the Field

The most important feedback did not come from benchmarks. It came from the field

workers who tested early builds under real conditions.

They told us that clarity under pressure is everything. A protocol that is technically accurate but visually confusing will be abandoned at the moment it is most needed. Every design decision in RescueNow — the checklist structure, the flowchart layout, the single-language rendering, the success confirmation — emerged from watching real people navigate real emergencies with an early version of the tool.

We are grateful to every frontline partner who gave us that feedback. This is their tool, shaped by their experience.

What RescueNow Actually Is

It is not a chatbot. It is not a diagnostic engine. It is a structured protocol assistant that runs offline, speaks your language, and hands you the next step when you need it most.

Because every village deserves a guardian. And every frontline worker deserves an expert sitting silently in their pocket.